# HerbHacks - AI Herbal Skin Care Solution

Complete Project Analysis Report

## 1. Project Overview

HerbHacks is an AI-powered holistic skin health tracking and diagnostic platform. The application allows users to analyze their skin for potential conditions (like acne, eczema, etc.), receive personalized Ayurvedic and herbal remedies based on those conditions, and track their skin health progress over time via a private cloud-synchronized dashboard. It also provides an OCR-based ingredient scanner to flag harmful chemicals in cosmetic products.

## 2. Technology Stack

Frontend:

- React (Vite) for fast UI rendering.
- Tailwind CSS for modern, responsive glassmorphism styling.
- Recharts for visualizing the skin health score progress.
- Supabase Authentication for login and session management.

Backend:

- FastAPI (Python) serving as the main backend router.
- TensorFlow (Keras) for AI skin disease classification (MobileNet/Custom CNN).
- MediaPipe & MTCNN for robust offline face detection and cropping.
- EasyOCR / Tesseract for ingredient scanning and text extraction.
- Supabase PostgreSQL for cloud database (scan_history, users).
- Supabase Storage for secure long-term image archiving.

## 3. Key Features List

1. AI Skin Analysis: Evaluates user-uploaded or live-captured face images to classify skin conditions.
2. Face Validation Safeguard: Prevents non-human objects from being evaluated by the AI.
3. Live Camera Tracking: Mediapipe detects and aligns the user's face in real-time.
4. Supabase Auth & History: Users have private dashboards securely displaying their historical charts.
5. Harmful Chemical Scanner (OCR): Users highlight ingredients on product labels to check toxicity.
6. Herbal Remedies DB: Provides organic Ayurvedic treatments mapped specifically to AI predictions.
7. Store Locator: Integrated mapping interface to locate nearby herbal/organic stores.

## 4. Data Architecture & Security

When a user performs a scan, the frontend delegates authentication tokens to the FastAPI backend. The backend first validates the image to ensure it contains a human face. If valid, the image is passed to the TensorFlow model which outputs a disease confidence score and matches it with a JSON database of remedies. Simultaneously, a background task uploads the image to a Supabase Storage Bucket, secures the private URL, and inserts the result block into the PostgreSQL 'scan_history' table. Supabase Row Level Security (RLS) ensures that when the user accesses their Dashboard later, they can only retrieve rows matching their unique cryptographic JWT signature.

## 5. Anticipated Judge Questions & Answers

Q1: How does your AI ensure it's actually analyzing skin and not a random object or wallpaper?
A1: We implemented strict safeguards using MediaPipe and MTCNN. Before our TensorFlow skin model runs, MTCNN scans the image. If no human face is detected with high confidence, the backend instantly rejects the upload, saving cloud compute resources and ensuring clinical data integrity.

Q2: How is user privacy and scan history secured?
A2: We use Supabase Authentication for secure JWT sessions. For the database, we implemented PostgreSQL Row Level Security (RLS). This cryptographic safeguard ensures that even though scans are efficiently saved to the cloud, the React frontend can only ever query and view the specific 'scan_history' rows that cryptographically match the logged-in user's ID.

Q3: What is the technical flow of the Live Camera feature?
A3: The React frontend accesses the webcam and runs MediaPipe's WASM face detector edge-side purely to ensure the user is perfectly aligned in the UI frame. Once aligned for 1 full second, it captures a base64 frame and sends it securely to our FastAPI backend. The backend runs the heavy TensorFlow classification, concurrently schedules an asynchronous background task to save to Supabase, and instantly returns the Ayurvedic remedies to the user for a lightning-fast experience.

Q4: How does the ingredient scanner work?
A4: We utilize Optical Character Recognition (OCR) on user photos of skincare product labels. The extracted text is normalized in Python and cross-referenced against our local dictionary of known harmful cosmetic

chemicals (e.g., Parabens, Sulfates). It strictly highlights toxic ingredients, promoting our holistic, herbal-safe ethos.